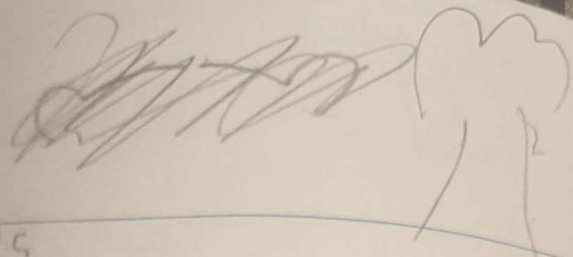


Shahir Vadda



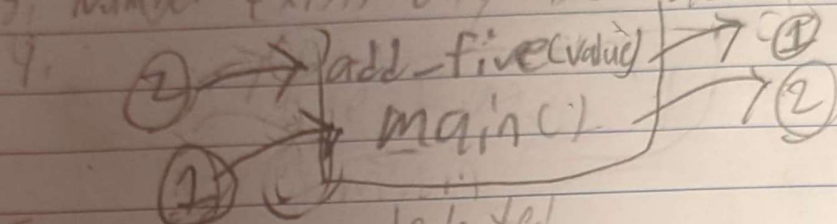
Mystery 1

1. Inside addFive(): 15

In main(): 10

2. Value doesn't change because by default Add five creates a copy of values inside the function and only changes that

3. Number exists only inside addFive



5. number is deleted

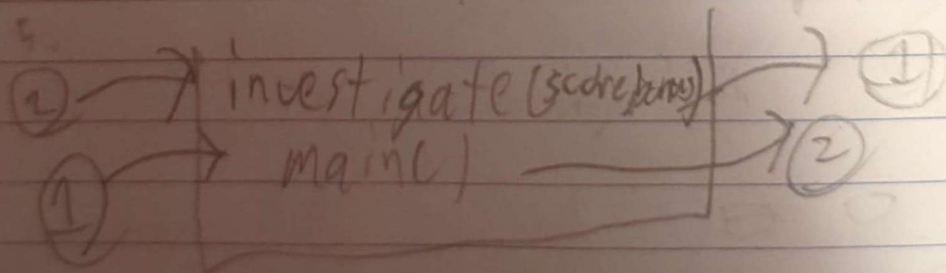
Mystery 2

1. Inside investigate() 15 5
15 5

2. score changes because investigate adds 5 to score while bonus is an unmodified constant

3. To make sure value isn't accidentally change its declared as a const. It's a reference, so it doesn't have to create another variable

4. It would result in a compilation error



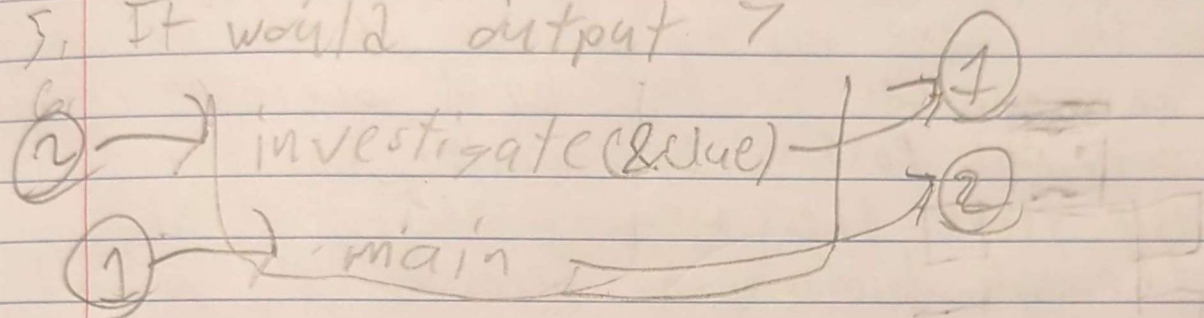
Mystery 3

1. The output is 14

2. To give the memory address to the pointer variable

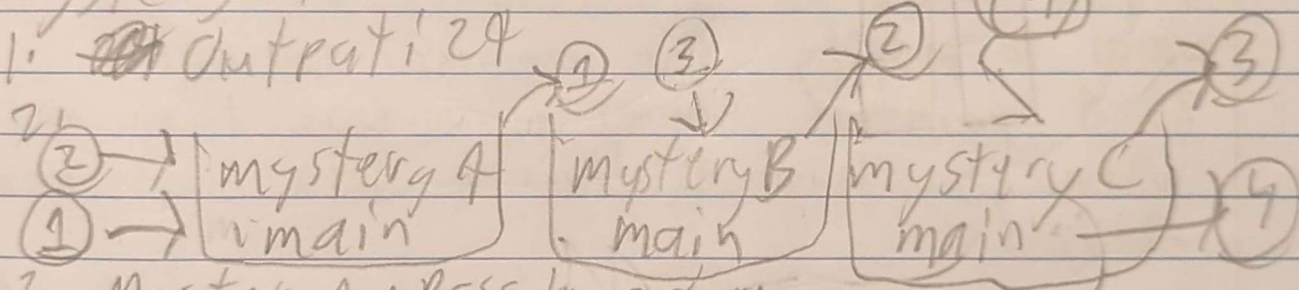
3. *evidence dereferences evidence and accesses the value in the memory address
4. nullptr makes sure the function actually points to something instead of having you crash when you try and dereference a non-existent ptr.

5. It would output 7



Mystery 4

1. ~~Output~~ Output: 24



3. Mystery A: Pass by copy
 Mystery B: Pass by Reference
 Mystery C: Pass by Pointer

4. Mystery B and Mystery C

5. mystery A is unable to directly modify the variable so it returns the value so you can set it to the variable

6. I would likely use Mystery B as I won't have to worry about null pointer although for readability I might use Mystery A to make it obvious I'm changing the value of the variable

Reflection

1. Passing by copy creates a copy of the variable so changes to the copy don't affect the original.

Passing by reference creates a variable with the same memory address as the original, thus any changes to the local variable changes the original. Passing by pointer has the variable store the memory address itself and through dereferencing accesses the original value.

2. You would use a reference when you just want to modify the original variable.

3. A pointer shows more use than references when you have to traverse data over memory addresses.

4. `const` prevents modification and only allows reading of the variable.

5. It shows us where and when certain variables exist and how changes in one part of the stack affects the next part.